Riphah International University, Lahore

Name: Bilawal Hussain

Sap ID: 61568

Course: Game Programming

Submitted to: Sir Hisham Zahid

Dota 2 Code Architecture Design

1. Game Overview

Dota 2 is a highly competitive, 5v5 multiplayer online battle arena (MOBA) game developed by Valve Corporation. The core gameplay loop centers on two teams—the Radiant and the Dire—battling to destroy the opposing team's central structure, known as the "Ancient." Players control distinct "Heroes" with unique abilities, leveling up, purchasing items, and engaging in complex real-time combat. Originally released on the Source engine, *Dota 2* was officially ported to Valve's heavyweight Source 2 engine in 2015 to support a 64-bit environment, Vulkan/Direct3D 11 rendering, and optimized mesh-based visibility algorithms.

2. High-Level Architecture

To maintain absolute competitive integrity, *Dota 2* relies on a highly deterministic, strictly server-authoritative architecture over a proprietary networking backbone.

2.1 Matchmaking and Network Routing System

Unlike games that expose dedicated servers directly to the public internet, *Dota 2* utilizes the Steam Datagram Relay (SDR). SDR acts as a massive virtual private gaming network capable of handling up to 140 million packets per second across Valve's global data centers. When a client finds a match via the central game coordinator, they are given a cryptographically signed ticket. Their traffic is securely routed through edge relays on the Valve backbone to the game server. This design completely obfuscates the game server's true IP address, rendering it functionally immune to Distributed Denial of Service (DDoS) attacks.

2.2 Network Transport: GameNetworkingSockets

The transport layer utilizes Valve's open-source ISteamNetworkingSockets library. This protocol builds a Reliable UDP (RUDP) implementation using an "ack vector" model inspired by Google QUIC. It automatically handles packet fragmentation, reassembly, and retransmission for reliable messages, while applying AES-GCM-256 encryption to every packet.

2.3 Game State Management

The server simulates the game in discrete time steps known as "ticks." The server processes all incoming user commands, executes the physics simulation, enforces game rules, and updates all object states. Because sending the entire world state every tick would overwhelm bandwidth, the server relies on "Delta Snapshots"—highly compressed packets that only contain the variables and coordinates that have mathematically changed since the client's last acknowledged update.

2.4 UI System (Panorama)

The user interface is powered by Panorama UI, an architecture deeply integrated into Source 2 that mimics modern web development paradigms. It abandons legacy systems (like Scaleform) in favor of XML files for layout (similar to HTML), CSS for visual styling, and JavaScript for

asynchronous event handling and data binding.

3. Code Structure

The codebase of *Dota 2* relies on a robust Entity-Component inheritance hierarchy natively written in C++, combined with an expansive Lua scripting API for abilities, logic, and artificial intelligence.

3.1 Classes and Entities

Every interactive object inherits from the core CBaseEntity class. The inheritance tree branches into more specialized classes, ultimately reaching CDOTA_BaseNPC, which is the foundational class governing all Heroes, creeps, and controllable units.

3.2 Combat, Abilities, and Modifiers

Abilities and status effects are organized via Data-driven text configurations (npc_abilities_custom.txt) or programmed directly in Lua utilizing classes like CDOTA_Ability_Lua and CDOTA_Modifier_Lua.

- **Abilities:** Expose strict state machine hooks such as OnSpellStart() (triggered when cast time ends and resources are consumed) and OnAbilityPhaseStart() (triggered as the cast animation begins).
- **Modifiers:** Handle buffs, debuffs, and auras. Modifiers hook directly into the engine's underlying C++ pipelines using declared properties. For example, declaring MODIFIER_PROPERTY_HEALTH_REGEN_CONSTANT injects a flat regenerative value into the entity's health pool per tick, while MODIFIER_PROPERTY_BASEDAMAGEOUTGOING_PERCENTAGE scales physical attack damage before armor mitigation.

3.3 Artificial Intelligence and Bots

Bot logic avoids simulating mouse clicks and instead directly queries the server's game state (restricted by Fog of War logic to prevent AI cheating). Scripting is modularized into specialized Lua files, such as ability_item_usage_generic.lua for combat logic, and mode_laning_generic.lua, where bots return floating-point desires (0.0 to 1.0) to dictate high-level strategies like pushing or defending.

3.4 Communication Between Systems

The client collects player inputs (mouse clicks, ability hotkeys) and batches them into UserCmd packets, sending them to the server at a defined command rate (cl_cmdrate). The server aggregates all inputs, ticks the simulation forward, and broadcasts Delta Snapshots back to all clients.

3.5 Diagram: Network and Data Flow

[Client Application]
| (Registers Input: Move Order)
v

[UserCmd Batching] --> [GameNetworkingSockets (AES-GCM-256)]
|
| (Encrypted UDP Packet via Steam Datagram Relay)
v
[Edge Relay Node] --> [Valve Backbone] --> [Authoritative Dota 2 Server]
|
[Server Simulation Tick]
| --> Applies Input to CDOTA_BaseNPC
| --> Triggers Lua API (OnAbilityPhaseStart, etc.)
| --> Updates Entity States
|
[Delta Snapshot Generation]
|
| (Encrypted UDP Delta Payload)
v
[Client Application] --> [Entity Interpolation] --> [Panorama UI Update]

4. Design Decisions

4.1 Strict Server Authority over Peer-to-Peer Lockstep

While many traditional Real-Time Strategy (RTS) games utilize a deterministic peer-to-peer "lockstep" model (where clients only send inputs to one another and simulate the game identically), *Dota 2* uses a Client-Server architecture. *Reasoning:* Lockstep architectures suffer from severe desynchronization if a single floating-point calculation differs between hardware configurations, and they easily enable map-hacking since every client must hold the entire game state in memory. The server-authoritative model prevents map-hacking (the server only sends data the client is permitted to see via Fog of War) and gracefully handles drop-in/reconnect functionality without forcing the client to re-simulate the entire match from tick zero.

4.2 Disabling Client-Side Prediction for Gameplay

In First-Person Shooters (FPS), the client immediately predicts player movement to hide network latency. However, *Dota 2* developers intentionally disabled client-side prediction for physical hero movement. *Reasoning:* If prediction were enabled in a dense MOBA environment, the client would guess pathfinding and collision resolutions. When the server inevitably corrected a wrong prediction (e.g., the player collided with an unseen entity in the Fog of War), the visual rubber-banding and unit teleporting would be highly disruptive. Valve chose 100% visual consistency over zero-latency responsiveness. The client immediately acknowledges the input via UI (e.g., green movement arrows appear instantly), but the physical character model does not move until the server has validated the command and returned the new state.

4.3 Sub-Tick Processing and Interpolation

Because the server simulates discrete ticks, visual rendering would naturally look jittery.

Reasoning: The client buffers incoming snapshots and smoothly interpolates the physical positions of models between the past two received states, creating perfectly smooth animations despite packet arrival times. To ensure high-fidelity input resolution without the massive CPU and bandwidth overhead of upgrading to 128-tick servers, modern Source 2 updates implemented Sub-tick processing. Client inputs are micro-timestamped per-frame, allowing the server to calculate the exact fractional millisecond a command occurred within the tick, providing extreme precision for spell casting and hit registration.

1.